CreativeGame: Toward Mechanic-Aware Creative Game Generation.

CreativeGame Team

Team Members (listed alphabetically): Hongnan Ma · Han Wang · Shenglin Wang · Tieyue Yin · Yiwei Shi · Yucong Huang

Team Leaders: Yingtian Zou · Muning Wen · Mengyue Yang

Institutions: University of Bristol · Shanghai Jiao Tong University · Shandong University · Nanjing University · Sreal AI

Project Page: yiweishi-cn.github.io/CreativeEvolutionGame

April 2026

Abstract

Large language models can generate plausible game code, but turning this capability into *iterative creative improvement* remains difficult. In practice, single-shot generation often produces brittle runtime behavior, weak accumulation of experience across versions, and creativity scores that are too subjective to serve as reliable optimization signals. A further limitation is that mechanics are frequently treated only as post-hoc descriptions, rather than as explicit objects that can be planned, tracked, preserved, and evaluated during generation.

This report presents **CreativeGame**, a multi-agent system for iterative HTML5 game generation that addresses these issues through four coupled ideas: a proxy reward centered on programmatic signals rather than pure LLM judgment; lineage-scoped memory for cross-version experience accumulation; runtime validation integrated into both repair and reward; and a mechanic-guided planning loop in which retrieved mechanic knowledge is converted into an explicit mechanic plan before code generation begins. The goal is not merely to produce a playable artifact in one step, but to support interpretable version-to-version evolution.

The current system contains 71 stored lineages, 88 saved nodes, and a 774-entry global mechanic archive, implemented in 6,181 lines of Python together with inspection and visualization tooling. The system is therefore substantial enough to support architectural analysis, reward inspection, and real lineage-level case studies rather than only prompt-level demos.

A real 4-generation lineage shows that mechanic-level innovation can emerge in later versions and can be inspected directly through version-to-version records. The central contribution is therefore not only game generation, but a concrete pipeline for observing progressive evolution through explicit mechanic change.

Contents

1 Introduction **4**
- 1.1 Contributions . . . 4
- 1.2 Report Scope . . . 5

2 Formal Foundations and Notation **5**
- 2.1 Game as a Rule-Organized Interactive System . . . 5
- 2.2 Meaningful Play and Learnability . . . 5
- 2.3 Mechanic as a Local Rule Structure . . . 6
- 2.4 Structural Creativity and Mechanic Delta . . . 6

3 System Architecture **6**
- 3.1 Reliability Mechanisms . . . 6
- 3.2 Iterations vs. Lineage Versions . . . 6
- 3.3 Mechanic-Guided Planning Loop . . . 7

4 CreativeProxyReward **8**
- 4.1 Formula . . . 8
- 4.2 Signal Sources . . . 8
- 4.3 Relation to the Formal Notation . . . 9
- 4.4 Why the Hard Gate . . . 9

5 Lineage-Aware Memory **10**
- 5.1 Memory Update Rule . . . 11
- 5.2 Why Lineage-Shared Memory . . . 11
- 5.3 Three-Layer Architecture . . . 11

6 Runtime Validator **11**
- 6.1 Motivation . . . 11
- 6.2 Tier 1: Deep Static Analyzer . . . 11
- 6.3 Tier 2: Browser Execution Check . . . 11
- 6.4 Pipeline Integration . . . 12

7 Implementation **12**

8 Case Study: Four Real 4-Version Evolutions **12**
- 8.1 Fireboy and Watergirl: From Dual-Avatar Traversal to Memory Relay . . . 12
- 8.2 Flappy Bird: From Obstacle Avoidance to Route Writing . . . 14
- 8.3 Happy Glass: From Drawing Supports to Programming Fluid State . . . 14
- 8.4 Plants vs. Zombies: From Static Lane Defense to Charged Interception . . . 14
- 8.5 Cross-Game Observations . . . 15
- 8.6 Implication for Reward Design . . . 15

9 Empirical Results **15**

10 Evaluation Protocol **16**

11 Related Work and Discussion **16**

12 Conclusion 17

1 Introduction

Generating creative interactive content (games) remains an unsolved problem for LLMs. A single LLM call given "make me a creative game" produces plausible-looking code that often fails at runtime, generic templates (Pong clones, basic shooters), no mechanism for accumulating what worked across generations, and subjective creativity scores that are hard to validate. More broadly, creativity research has long emphasized both the difficulty of judging creativity and the importance of evaluative perspective [1, 2]. The fundamental difficulty in our setting is therefore that **creativity is open-ended, hard to evaluate, and hard to optimize**, yet a measurable optimization signal is required for iterative improvement.

CreativeGame addresses this problem through five tightly coupled design choices. First, creative game generation is decomposed into 7 logical agents (10 executable roles including 4 code-generation sub-agents), each with a focused prompt and parameter profile. Second, subjective LLM scoring is replaced with a **CreativeProxyReward** composed of weighted proxy signals and gating conditions, so that only a minority of the total signal depends directly on LLM judgment. Third, the system introduces a **lineage-aware memory** in which all forks within a lineage share a common learned memory pool, allowing experience to accumulate across versions; this design is directly informed by recent MemRL-style work on runtime reinforcement learning over episodic memory [3]. Fourth, a **runtime validator** performs deep static analysis and optional browser execution to catch bugs that model-side evaluation may miss. Fifth, **mechanics are promoted to explicit planning objects**: planner-time mechanic retrieval produces a structured mechanic plan that can later be compared with realized mechanics and stored in evolution records. Each generation call runs up to 3 total iterations (1 initial generation followed by up to 2 refinement passes), and only the final state is saved as a lineage node. This framing is important because the project is intended to support not only generation, but interpretable version-to-version evolution.

1.1 Contributions

1. A formulation of iterative game generation in which mechanics are treated as explicit planning and evaluation objects, rather than only as retrospective descriptions of generated content (Sections 2 and 3.3).
2. A *CreativeProxyReward* whose dominant signals are deterministic Python-side measurements, combining mechanic realization, structural change, novelty, and runtime validation while reducing dependence on unconstrained LLM judgment (Section 4).
3. A lineage-aware memory architecture that shares experience across versions within a lineage while preserving isolation across lineages, making iterative accumulation possible without collapsing all generations into a single global memory pool (Section 5).
4. Integration of runtime validation directly into the generation loop as both a repair trigger and a reward gate, including a lightweight static analyzer and optional browser-based execution checks (Section 6).
5. A fully self-contained implementation of the complete pipeline, together with real-lineage evidence showing concrete mechanic-level innovation across iterations (Sections 7 and 8).

1.2 Report Scope

This report describes the current CreativeGame system. Architectural descriptions and system statistics are aligned with the implemented pipeline and its stored generation data.

2 Formal Foundations and Notation

This report adopts a notation system aligned with the current project concept documents on game definition and mechanic definition. The aim is to keep the report, the code interpretation, and future evaluation criteria consistent.

2.1 Game as a Rule-Organized Interactive System

We represent a game as

$$G = (P, S, A, T, O, F, K, W, U, \Phi, C, R, M), \tag{1}$$

where P denotes decision-bearing agents, S the state space, A the action space, T the transition rule, O the observation structure, F the feedback mapping, K the resource/constraint structure, W the outcome structure, U the preference ordering over outcomes, Φ the challenge structure, C the content layer, R the representation layer, and M the meta layer.

Following the project definition, we distinguish:

$$G_{\text{core}} = (P, S, A, T, O, F, K, W, U, \Phi), \tag{2}$$

$$G_{\text{support}} = (C, R, M). \tag{3}$$

Changes to G_{core} count as structural game changes, while changes confined to G_{support} are cosmetic or presentation-level changes. This distinction is important throughout the report: the CreativeGame pipeline is intended to reward structural variation rather than pure reskinning.

2.2 Meaningful Play and Learnability

The concept documents define two validity predicates over the assembled game system. This emphasis on meaningful play is broadly compatible with classic game-design accounts that center rules, player interpretation, and consequence [4, 5].

$$\Psi(G) \in \{0, 1\}, \tag{4}$$

$$\Lambda(G) \in \{0, 1\}, \tag{5}$$

where $\Psi(G)$ is the meaningful-play condition and $\Lambda(G)$ is the learnability condition. Intuitively, $\Psi(G) = 1$ means that action outcomes are both discernible and integrated into the broader system, while $\Lambda(G) = 1$ means that the game contains exploitable regularities such that non-random strategy can improve expected outcomes.

Accordingly, a valid game is one whose core structure is complete and which satisfies both predicates:

$$G \text{ is a valid game} \iff G_{\text{core}} \text{ is structurally complete} \wedge \Psi(G) = 1 \wedge \Lambda(G) = 1. \tag{6}$$

2.3 Mechanic as a Local Rule Structure

Within this report, a game mechanic is not treated as a theme tag or content label, but as a local rule structure inside the game system. This is broadly compatible with direct attempts to define game mechanics in terms of player interaction methods and rule-bearing game structures [6]. We use the compact formalization

$$m = (\Delta A, \Delta T, \Delta O, \Delta F, \Delta K, \Delta W), \tag{7}$$

meaning that a mechanic is identified by the stable way it changes at least one of the structural layers most relevant to play: action space, transition logic, information structure, feedback relation, resource constraints, or goal progression.

To distinguish existence from quality, we also use three mechanic-level scores:

$$m \mapsto (E_m, I_m, V_m), \tag{8}$$

where E_m denotes mechanic existence, I_m mechanic importance, and V_m showcase value. In the concept documents, E_m acts as a gate for whether a candidate should count as a mechanic at all, I_m captures how strongly it matters to decision structure and the core loop, and V_m captures whether the mechanic can be clearly observed in a short evaluation window.

2.4 Structural Creativity and Mechanic Delta

Given a parent game G and a generated variant G', structural creativity should ideally be attributed to changes in the core rule-bearing structure rather than to support-layer variation alone. We therefore use the notion of mechanic delta in the following sense:

$$\delta(G, G') = \{m \mid m \in G'_{\text{core}} \setminus G_{\text{core}}\} \cup \{m \mid m \in G_{\text{core}} \setminus G'_{\text{core}}\}. \tag{9}$$

In implementation, the live system approximates this idealized delta through extracted mechanic sets and related reward terms. Nevertheless, Equations 1–9 provide the formal reference used throughout the report whenever it discusses structural change, mechanic preservation, novelty, and planned-vs-realized mechanics.

3 System Architecture

The system contains 7 logical agents, with the generation stage composed of 4 internal sub-stages (Skeleton, Feature, Visual, Refinement), yielding 10 distinct executable roles. Table 1 summarizes the configuration.

3.1 Reliability Mechanisms

Three layers of error recovery are used: repeated model calls, stage-wise fallback, and tolerant final formatting. Together these reduced the pipeline failure rate from $\sim$10% to $<$2%.

3.2 Iterations vs. Lineage Versions

Each generation call runs up to 3 total iterations: 1 initial generation followed by up to 2 refinement passes. Only the final state is saved as a lineage node. User-visible "v1/v2/v3/v4" labels refer to *separate generation calls*, not internal refinement passes within one call.

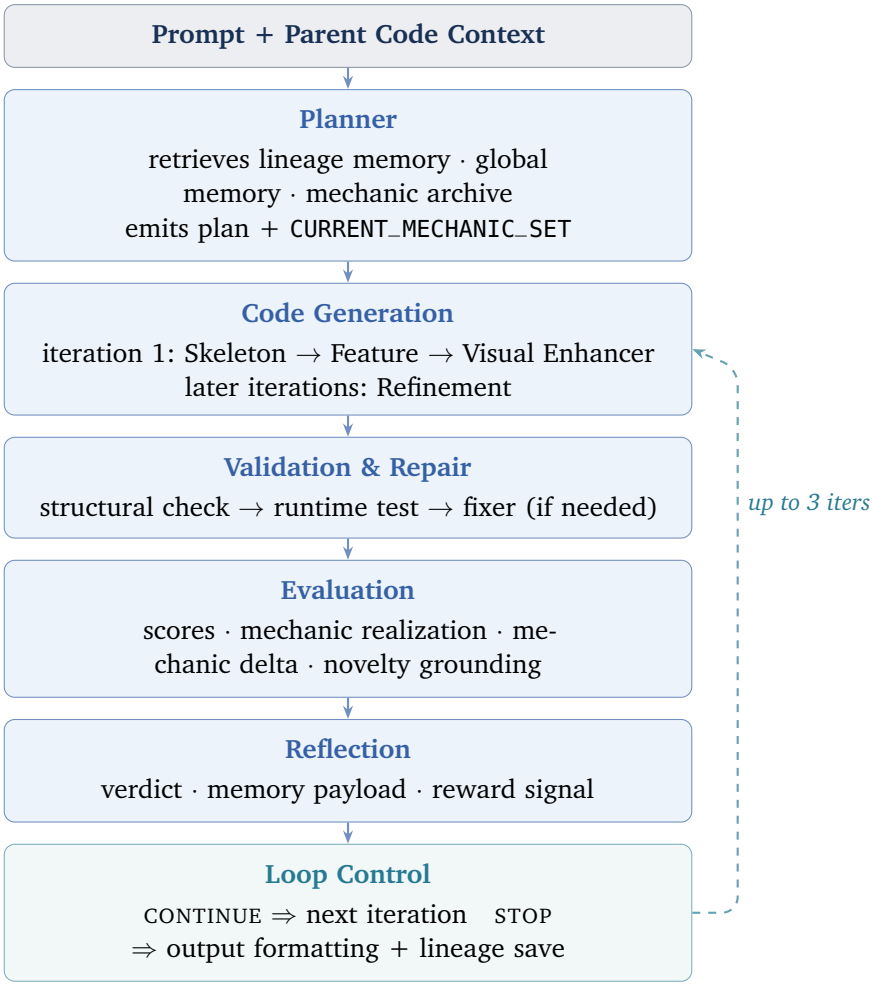

Figure 1: Code-grounded overview of the implemented pipeline (`pipeline.py`, `agents.py`). The dashed feedback arc indicates the refinement loop: after CONTINUE, control returns to the Code Generation stage for up to 2 further passes before STOP triggers output formatting and lineage save.

3.3 Mechanic-Guided Planning Loop

The system includes an explicit mechanic layer between prompt interpretation and code generation. Concretely, the planner receives retrieved mechanic-library context, emits a labeled mechanic plan, and the orchestration loop stores that structure for each iteration. The same plan is then appended to later evaluation and reflection stages, enabling planned-vs-realized mechanic comparison and mechanic-aware memory writing.

This changes the interpretation of the system in an important way. Mechanics are treated not merely as archive entries or post-hoc descriptors, but as explicit control variables: the planner can state which mechanics should be preserved, added, removed, or recombined before code generation begins. In the notation of Section 2, the pipeline therefore moves beyond post-hoc content description toward explicit planning over candidate local rule structures m and their intended changes to G_{core}. The system also exposes planning and evaluation records in an inspectable form, making version-to-version mechanic change directly observable.

Table 1: Role specifications in the current system. Generation parameters and token budgets are tuned per role; the Generation stage is split into four sequential sub-roles.

Role	Sub-role	Temp.	Token budget
Planning	—	0.7	12,000
	Skeleton	0.7	4,096
	Feature	0.8	16,000
	Visual	0.8	20,000
	Refinement	0.7	24,000
Repair	—	0.3	20,000
Runtime Validation	—	*n/a*	*n/a*
Evaluation	—	0.2	4,000
Reflection	—	0.3	3,000
Output Formatting	—	0.2	5,000

4 CreativeProxyReward

Prior work uses LLM-based scoring as the primary reward signal. We observe three problems: (i) score saturation (GPT-class models default to 7/10 regardless of input), (ii) no verifiable improvement (a 7→8 change is not statistically distinguishable from noise), and (iii) Goodhart's-law risk (optimizing for LLM judgment leads to outputs that "sound creative" without being mechanically novel).

4.1 Formula

The CreativeProxyReward consists of 7 weighted signal terms and 2 gating conditions:

$$\begin{aligned} \textit{Reward} = \; & +0.20 \cdot \textit{MechanicRealization} \\ & +0.25 \cdot \textit{StructuralMechanicChange} \\ & +0.20 \cdot \textit{RelativeMechanicNovelty} \\ & +0.15 \cdot \textit{LLM_Creativity} \\ & +0.10 \cdot \textit{RuntimePlayability} \\ & -0.15 \cdot \textit{CosmeticOnlyPenalty} \\ & -0.10 \cdot \textit{RegressionPenalty} \end{aligned} \tag{10}$$

subject to two gating conditions (Figure 3):

$$\textit{Reward} \leftarrow \begin{cases} 0.25 \cdot \textit{Reward} & \text{if } \textit{PlayabilitySanity} < 0.6 \quad \text{(soft gate)} \\ 0.5 \cdot \textit{Reward} & \text{if not } \textit{runtime_test_passed} \quad \text{(hard gate)} \end{cases} \tag{11}$$

4.2 Signal Sources

MechanicRealization measures whether the generated game actually realizes the planned mechanics. In the formal view, this acts as an implementation proxy for whether the intended mechanic-level changes to G_{core} are realized in the generated artifact. *StructuralMechanicChange*

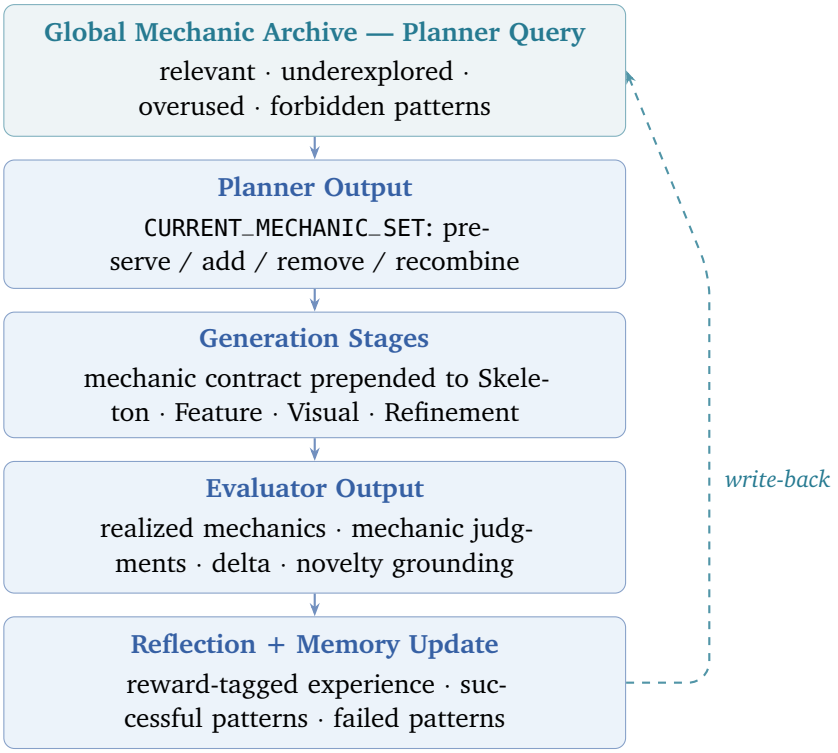

Figure 2: Mechanic-centered feedback loop. Mechanics are retrieved *before* planning (top), converted into an explicit generation contract, compared against realized mechanics in evaluation, and conditionally written back into the archive (dashed arc) after reflection.

is computed from added, modified, and removed mechanics together with an overall structural-change score. *RelativeMechanicNovelty* is grounded against the global mechanic archive, which currently contains 774 entries. *LLM_Creativity* is $(\text{score} - 3)/7$ clamped to $[0, 1]$. *RuntimePlayability* is the tester score (Section 6); despite the historical field name, we interpret it here as a proxy for runtime robustness and execution quality rather than a direct measure of human-perceived fun. *CosmeticOnlyPenalty* penalizes outputs with negligible structural change, and *RegressionPenalty* captures missing executable core features such as canvas setup, game loop, or input handling.

4.3 Relation to the Formal Notation

The current reward implementation should be understood as an engineering proxy rather than a complete realization of the formal framework in Section 2. In the formal view, structural creativity should respond to changes in G_{core}, to the appearance or preservation of local rule structures m, and ideally to their contribution to meaningful play $\Psi(G)$ and learnability $\Lambda(G)$. The present implementation approximates this target through extracted mechanic deltas, mechanic realization, archive novelty, runtime robustness, and penalties for purely cosmetic or regressive outputs. The distinction matters: the notation defines the target semantics, whereas the current implementation provides an operational approximation.

4.4 Why the Hard Gate

The runtime hard gate (*Reward* $\times$ 0.5 if test fails) is critical because it prevents the system from rewarding "creative-looking" games that do not actually run. Unlike LLM-based signals, this

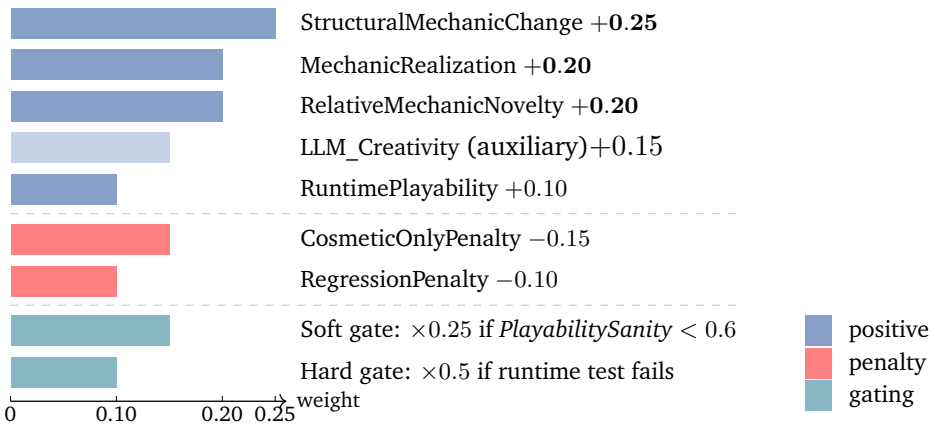

Figure 3: CreativeProxyReward signal weights (scale: 3.5 cm = 0.25). The three mechanic-grounded signals account for 65% of the maximum positive weight; LLM judgment contributes only 15%. Gating conditions are applied multiplicatively after summing the weighted terms.

signal cannot be gamed by LLM optimization because it executes the actual code. In the current formulation, *LLM_Creativity* remains auxiliary, while mechanic realization, structural change, novelty, and runtime robustness dominate the score.

5 Lineage-Aware Memory

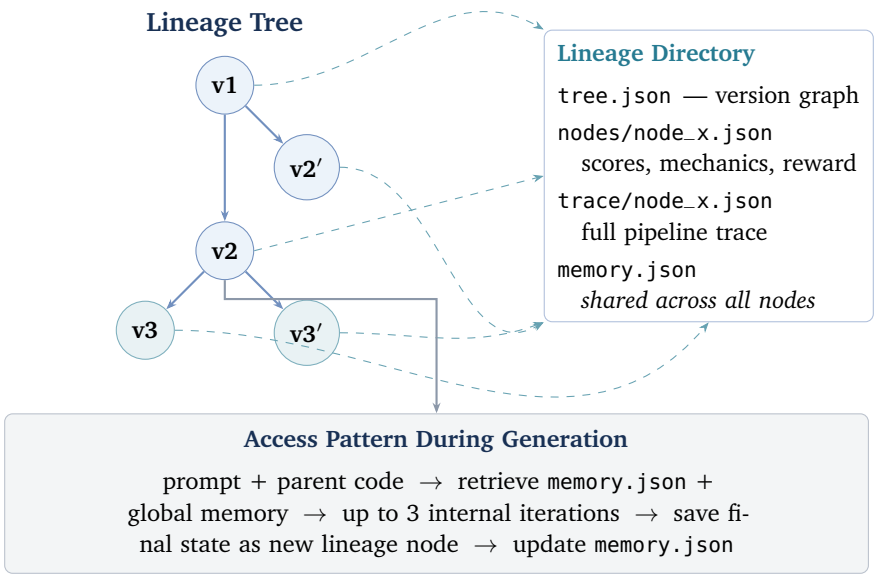

Figure 4: Lineage-level storage and memory sharing (`memory/manager.py`). All nodes share `memory.json` (dashed arcs); lineages are isolated from each other. Internal refinement iterations occur *inside* one generation call; user-visible versions (v1, v2, ...) are stored as separate tree nodes.

We organize game versions as a **lineage tree**, where each node is one generation and edges represent parent-child relationships. Memory is **shared across all nodes in a lineage but isolated across lineages**. Version structure, node-level outputs, memory state, and inspection records are stored together at the lineage level.

5.1 Memory Update Rule

Memory items are represented as tuples over intent, representation, value estimate, and visit count. After each iteration, the stored value is updated by exponential averaging, $q' = (1-\alpha)q + \alpha r$ with $\alpha = 0.3$ and reward $r \in [-1, 1]$. Retrieval combines semantic similarity and learned value, balancing reuse of relevant past experience against exploitation of historically successful patterns.

5.2 Why Lineage-Shared Memory

Two alternatives were considered: per-node memory (each node starts from zero, defeating the purpose of MemRL [3]) or per-lineage shared memory (all nodes accumulate experience). We chose the latter because (i) the entire purpose of MemRL is accumulation, (ii) cross-lineage isolation is preserved at the directory level, and (iii) a global creativity-rules layer handles truly universal patterns.

5.3 Three-Layer Architecture

Layer 1: per-lineage learned memory. *Layer 2*: cross-lineage memory resources, including creativity rules, a game pool, and the mechanic archive. *Layer 3*: transient pipeline context for the current generation.

Layer 2 is not merely a passive novelty baseline. The planner queries the global archive for relevant, underexplored, overused, and disfavored mechanics. After reflection, successful generated mechanics can be written back into the archive, creating an initial feedback loop between retrieval and write-back. At the notation level, the archive is best interpreted as an evolving memory over candidate mechanic objects m and partial approximations to their effects on G_{core}.

6 Runtime Validator

6.1 Motivation

LLMs frequently produce code that *looks correct* (passes structural keyword checks) but *does not run*. Common failure modes include game-loop functions that are defined but never called, `requestAnimationFrame` references with no recursive self-call, unbalanced braces, missing canvas context, and DOM access before `window.onload`.

6.2 Tier 1: Deep Static Analyzer

Always runs, no dependencies, <10 ms per game. Performs 9 checks: brace/paren/bracket balance, game loop invocation (`requestAnimationFrame()` actually called, not just defined), recursive loop self-call, canvas context obtained, input listener attached, init-on-load presence, render-call presence, state-update presence. Each error reduces score by 0.20, each warning by 0.05.

6.3 Tier 2: Browser Execution Check

Optional. If browser automation is available, the system launches a headless browser, loads the HTML, waits for canvas paint, sends basic inputs, and collects console errors. Returns

playable = True if no console errors occur and the canvas paints successfully. Otherwise, the system degrades gracefully to Tier 1.

6.4 Pipeline Integration

The runtime validator is invoked after code generation and before evaluation. If the test fails, a repair stage is invoked with the runtime errors as context, after which the game is re-tested. The runtime score becomes the 7th proxy signal in the reward formula and a hard gate.

7 Implementation

The current system is implemented as a self-contained Python pipeline. The implementation comprises 6,181 lines of Python, excluding generated data, presentation assets, and virtual-environment files. The system directly implements orchestration, memory access, reward computation, runtime validation, mechanic retrieval, lineage recording, and inspection interfaces within a single codebase.

This implementation choice matters because it keeps the full control flow visible and inspectable. The system can expose intermediate mechanic plans, integrate Python-native validation and reward logic, and store lineage records in a format aligned with the analysis questions of this report. The implementation is therefore not just a delivery mechanism for prompts, but part of the experimental contribution itself.

8 Case Study: Four Real 4-Version Evolutions

To illustrate the system in operation, we analyze four real 4-version game demos extracted from the current project website: *Fireboy and Watergirl*, *Flappy Bird*, *Happy Glass*, and *Plants vs. Zombies*. Each demo exposes a complete v1–v4 sequence and is useful for a different reason: platform coordination, one-button arcade control, physics-puzzle routing, and lane-defense planning. Together they provide a clearer picture of how the system changes its understanding of a source game across generations. Figure 5 shows all sixteen versions rendered simultaneously in a live browser grid, with each game running an injected demo bot; the screenshot captures representative mid-play states across all four lineages.

8.1 Fireboy and Watergirl: From Dual-Avatar Traversal to Memory Relay

The v1 game (*Echo Relay Temple*) already departs from standard dual-character co-op by turning the inactive body into part of the puzzle: one avatar can be parked to power an aura crystal while the active avatar continues traversal. At this stage, however, the game still largely reads as an extended elemental platform puzzle with one additional relay device.

The main shift happens in v2 and becomes clearer in v3–v4. In *Relay Glyph Temple* and *Relay Echo Temple*, swapping no longer functions only as control transfer; it creates a replay ghost that can trigger sensors and open routes. A gravity glyph is then introduced so that the recorded replay inherits a transformed physical rule rather than merely replaying motion. By v4 (*Memory Relay*), the system has a much sharper understanding of the game concept: the puzzle is no longer “control two elemental bodies” but “construct a living circuit out of parked states, ghost replays, and gravity-imprinted memory.” This is a good example of source-game understanding becoming more mechanic-explicit over time.

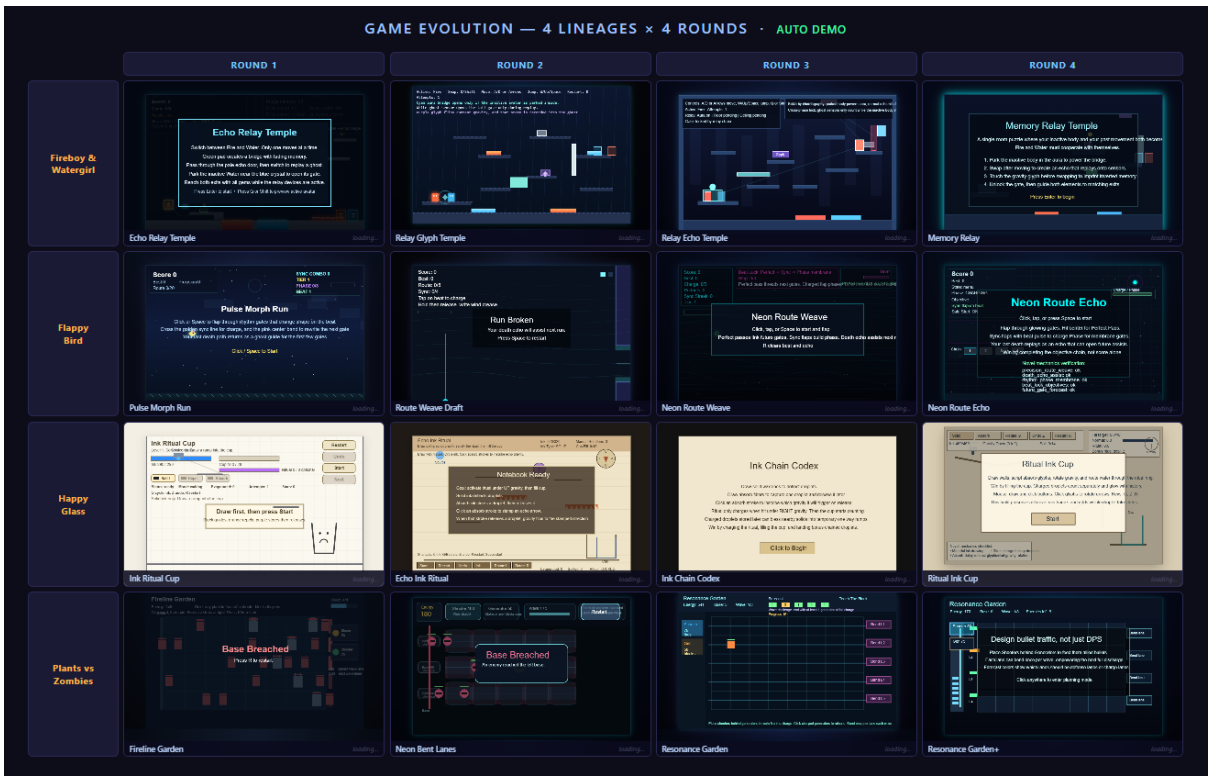

Figure 5: All four 4-round evolution lineages displayed as an auto-demo grid. Each column is one generation round (Round 1–4); each row is one source-game lineage. Games run live in browser with injected demo bots; this screenshot captures representative mid-play states.

Table 2: Four representative 4-version evolution sequences. Later versions typically *reinterpret* the source game around a more explicit mechanic contract rather than only adding polish.

Source Game	Initial understanding (v1)	Mechanic reinterpretation (v2–v4)
Fireboy & Watergirl	Character-switching elemental platform puzzle with relay and timing	Reinterpreted as **memory relay**: parked bodies, replay ghosts, and gravity-imprinted echoes become the core puzzle logic
Flappy Bird	One-button obstacle dodging with precision timing	Reinterpreted as **route authoring**: perfect passes rewrite future gates, death echoes assist later runs, and beat-synced phase windows change collision logic
Happy Glass	Draw-to-route fluid puzzle with physical barriers	Reinterpreted as **programmable fluid logic**: absorb strokes store droplets, release events rewrite gravity, and ritual state changes how fill is counted
Plants vs. Zombies	Resource-aware lane defense with plant placement and wave management	Reinterpreted as **interception planning**: generators intentionally block allied shots, store charge, and discharge through lane bending / refraction windows

8.2 Flappy Bird: From Obstacle Avoidance to Route Writing

The v1 flier (*Pulse Morph Run*) still preserves the recognizably simple Flappy Bird interaction loop: one button, vertical impulse, and continuous obstacle traversal. Its main novelty is that gates mutate with rhythm-like timing, so the game is already more structured than a plain obstacle dodger.

The later versions reinterpret what "passing a gate" means. In v2 (*Crease Choir*), clean passes can author links into future gates, death traces become ghost echoes that assist later runs, and rhythm timing can charge a temporary phase state. In v3 (*Neon Route Weave*) and v4 (*Neon Route Echo*), this understanding becomes much more coherent: perfect passes are not only scored events, but causal edits to the next route; the last failed run leaves an echo that can open assisted passages; and rhythm is tied to membrane-phasing rather than only cosmetic pulse. The genre understanding therefore shifts from reaction-based survival to a lightweight planning-and-rewrite loop in which the player's past trajectory actively shapes the near future.

8.3 Happy Glass: From Drawing Supports to Programming Fluid State

The v1 sequence (*Ink Ritual Cup*) begins close to a recognizable Happy-Glass-style template: the player draws lines to route droplets into a cup while avoiding loss. Even here, the system experiments with multiple ink materials and introduces a ritual checkpoint, but the dominant reading is still "physics puzzle with drawn barriers."

By v2 (*Echo Ink Ritual*), absorb strokes can store a droplet and later release it while changing gravity to an inscribed direction. This is the crucial conceptual step: the drawn line becomes not only geometry but a delayed rule trigger. In v3 (*Ink Chain Codex*), the game further adds relay interactions in which charged droplets can bless nearby solids and propagate behavior through neighboring strokes. In v4 (*Ritual Ink Cup*), the overall interpretation becomes cleaner and more legible: solid ink shapes paths, absorb ink scripts delayed state transitions, gravity rotations are limited strategic resources, and ritual-charged droplets are counted separately from normal fill. The source-game idea is therefore re-understood as a small programmable physics language rather than as a pure drawing puzzle.

8.4 Plants vs. Zombies: From Static Lane Defense to Charged Interception

The v1 lane-defense game (*Fireline Garden*) still reads closest to the original source structure. There are rows, wave previews, plant placement, and lane-local combat. The main novelty is already visible, though: energy generators can physically block allied fire, so resource production and shooting geometry interfere with each other rather than remaining cleanly separated.

That interaction becomes the center of the design in later versions. In v2 (*Neon Bent Lanes*), an entire lane can be bent once per wave, rewriting projectile travel and enemy-path geometry. In v3 (*Resonance Garden*), blocked allied shots are explicitly stored as overcharge in generators and later released as stronger resonance attacks, creating a deliberate "friendly obstruction as preparation" mechanic. In v4, this logic is made more strategically readable by forecast-guided lane planning and by framing bends as a once-per-wave refractive discharge window. The understanding of Plants-vs.-Zombies-like play thus shifts from "place units to stop waves" to "plan which lanes should defend directly and which lanes should absorb fire now in order to release stronger refracted attacks later."

8.5 Cross-Game Observations

Figure 5 provides an animated summary of all four lineages side-by-side; the discussion below unpacks each one. Across all four sequences, three common patterns emerge.

1. The most interesting changes are **mechanic reinterpretations**, not only visual polish. Later versions tend to re-assign meaning to an existing action: swap becomes memory writing, passing a gate becomes route editing, drawing becomes rule scripting, and blocking allied fire becomes intentional charge storage.
2. The system often moves from **surface genre mimicry to explicit causal structure**. Early versions preserve the recognizable shell of the source game, while later versions more clearly expose what hidden rule the new variant is really about.
3. The four examples also show that **game understanding changes by domain**. In platforming, the shift is toward state coordination; in arcade flying, toward future-route shaping; in physics puzzles, toward programmable matter; and in lane defense, toward forecast-based energy planning.

8.6 Implication for Reward Design

These case studies illustrate why mechanic-aware records are necessary. If evaluation only asked whether the output still "looks like" Flappy Bird or Plants vs. Zombies, many of the most interesting changes above would be collapsed into style variation. By contrast, when the system records intended mechanic sets, realized mechanics, and mechanic deltas, it becomes possible to describe evolution in terms of changing rule-bearing structure. This is precisely the level at which the four lineages above become interpretable.

9 Empirical Results

Generated data. The system contains 71 stored lineages: 9 multi-node lineages (up to depth 4) and 62 single-node lineages, for 88 saved nodes overall. The global mechanic archive contains 774 entries, and the summed token count recorded in saved nodes exceeds 4.5×10^6.

Per-stage computational budget. The visual generation stage is the largest consumer ($\sim$34% of total), followed by evaluation ($\sim$27%), feature generation ($\sim$18%), skeleton generation ($\sim$9%), planning ($\sim$8%), and reflection ($\sim$4%). The visual stage dominates because it adds presentation detail and animation on top of an already-substantial game body.

Reliability. After implementing the retry-and-fallback mechanisms in Section 3, pipeline success rate is $>$98%, with empty-output recovery rate $>$95% within 3 retries.

Score distribution. Across all generated games' final iteration, average creativity is $\sim$7.0/10, average evaluator *playability* score is $\sim$6.5/10, and average overall is $\sim$6.2/10. The terminology here follows the current evaluator schema, but the field should be interpreted as a coarse proxy for usability and functional completeness rather than as a validated measure of player enjoyment. These scores are also subject to LLM scoring saturation (Section 4) and are not validated against human judgment.

10 Evaluation Protocol

Prompt dataset. Game prompts were drawn from an internal prompt library spanning multiple genre categories. Each prompt was used to generate one lineage, with up to three internal refinement iterations.

Representative source games for mechanic coverage. For simple, legible mechanic exemplars, the current project also maintains a strict reference table of 252 web games with compressed genre and tag vocabularies. Four especially useful anchors for report discussion are *Flappy Bird* (single-input reaction loop), *Fireboy and Watergirl* (dual-character co-op platforming), *Happy Glass* (draw-to-shape physics puzzle), and *Plants vs. Zombies* (resource-aware lane defense). These examples are useful not because the system clones them directly, but because they span very different mechanic structures while remaining simple enough to explain and retrieve.

Generation settings. Maximum iterations 3; planning temperature 0.7; evaluation temperature 0.2; runtime validation enabled; optional browser-based execution check; memory retrieval top-k = 5.

Measurement definitions.

- *Generation time* is measured from the start of planning to the completion of final formatting, including any internal repair loops.
- *Computational usage* includes all model consumption from planning, generation, evaluation, and reflection, including retries and fallback executions.
- *Evaluator playability* follows the current evaluator field name, but operationally it should be read as a coarse usability/completeness proxy. Runtime robustness is measured separately by the validator score combining static analysis and (if available) browser execution.

Model configuration. The system uses contemporary large-language-model backends. Some lineages were generated with Kimi (server) and others with GPT-class models (local). Python 3.12, Ubuntu 24.04 (server) / Windows 11 (development).

11 Related Work and Discussion

Multi-agent code generation. Frameworks such as ChatDev, MetaGPT, and AgentVerse decompose software generation into role-based agents [7, 8, 9]. CreativeGame follows the general intuition of role specialization, but organizes it around a fixed iterative pipeline for game generation, testing, evaluation, reflection, and memory writing.

LLM creativity evaluation. Prior work on creativity theory and assessment emphasizes both the difficulty of defining creativity beyond simple novelty and the importance of who gets to judge it [1, 2]. More recent work on LLM-as-a-judge has shown both the promise and the limitations of model-based evaluation in open-ended settings [10]. Our CreativeProxyReward differs from judge-heavy LLM evaluation by minimizing the LLM judgment component to a single auxiliary signal and grounding the primary signals in deterministic Python computations.

Formal game and mechanic structure. Relative to game-studies-inspired formalizations, the present project adopts a structural view of games as rule-organized systems G and mechanics as local rule structures m. This framing is important because it clarifies what should count as creativity in the pipeline: changes to G_{core} rather than merely to G_{support}, and mechanic-level changes rather than purely thematic or cosmetic variation. In this respect, the report is closest to game-design accounts that foreground rules, interaction, and meaningful consequence [4, 5].

Game design foundations. Foundational game-design texts consistently stress the interaction between rules, player understanding, iteration, and the design of meaningful experience [4, 5, 11]. The present project extends these concerns into an automated setting by making mechanic planning, realization, and cross-version change explicit objects inside the generation loop.

Memory-augmented agents. Memory-augmented agent systems introduce persistent experience into sequential decision processes. Recent MemRL-style work makes this idea explicit through runtime reinforcement learning over episodic memory [3]. We adapt this general direction to creative generation, with the design choice of *lineage-scoped* sharing rather than per-task isolation, motivated by the desire for cross-version experience accumulation.

Runtime validation in code generation. Evaluation work on code-generating language models has strongly emphasized execution-based correctness [12]. Our innovation is integrating runtime validation as both a reward signal and a repair trigger within the multi-agent pipeline, with a graceful degradation path when richer execution checks are unavailable.

12 Conclusion

We presented CreativeGame, a multi-agent system for iterative creative game generation. Its central contributions are a CreativeProxyReward whose primary signals are programmatic, a lineage-aware memory that enables cross-version experience accumulation, runtime validation integrated as both a reward signal and a hard gate, and a mechanic-guided planning layer in which retrieved archive knowledge is converted into an explicit mechanic plan. The real lineage case study demonstrates that version-to-version mechanic evolution can be recorded, inspected, and discussed in concrete structural terms.

Taken together, the system shows that creativity in game generation can be approached as an inspectable engineering problem: mechanics can be planned explicitly, evaluated structurally, stored across generations, and followed through iterative evolution rather than treated only as a final subjective impression.

References

[1] J. C. Kaufman and J. Baer, "Beyond new and appropriate: Who decides what is creative?" *Creativity Research Journal*, vol. 24, no. 1, pp. 83–91, 2012.

[2] V. P. Glăveanu and R. A. Beghetto, "Creative experience: A non-standard definition of creativity," *Creativity Research Journal*, vol. 33, no. 2, pp. 75–80, 2021.

[3] S. Zhang, J. Wang, R. Zhou, J. Liao, Y. Feng, W. Zhang, Y. Wen, Z. Li, F. Xiong, Y. Qi, B. Tang, and M. Wen, "Memrl: Self-evolving agents via runtime reinforcement learning on episodic memory," 2026. [Online]. Available: https://arxiv.org/abs/2601.03192

[4] K. Salen and E. Zimmerman, *Rules of Play: Game Design Fundamentals*. MIT Press, 2003.

[5] J. Schell, *The Art of Game Design: A Book of Lenses*. Elsevier/Morgan Kaufmann, 2008.

[6] M. Sicart, "Defining game mechanics," *Game Studies*, vol. 8, no. 2, 2008. [Online]. Available: https://www.gamestudies.org/0802/articles/sicart

[7] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, "Chatdev: Communicative agents for software development," *arXiv preprint arXiv:2307.07924*, 2023. [Online]. Available: https://arxiv.org/abs/2307.07924

[8] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=VtmBAGCN7o

[9] W. Chen, Y. Su, J. Zuo, C. Yang, C. Yuan, C. Qian, C.-M. Chan, Y. Qin, Y. Lu, R. Xie *et al.*, "Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors in agents," *arXiv preprint arXiv:2308.10848*, 2023.

[10] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," *arXiv preprint arXiv:2306.05685*, 2023. [Online]. Available: https://arxiv.org/abs/2306.05685

[11] R. Koster, *A Theory of Fun for Game Design*. Paraglyph Press, 2005.

[12] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021. [Online]. Available: https://arxiv.org/abs/2107.03374